

UNIVERSIDAD ICESI

Arquitectura de Software

SISTEMA DISTRIBUIDO SITM-MIO

**Diseño arquitectónico, implementación de patrones y
experimentación con procesamiento histórico distribuido**

Informe técnico integral del proyecto

Integrante	Código
Fabio Felipe Murillo Rivas	A00401131
Valentina Arana Babativa	A003999932
Samuel Chapaval Ricci	A00398858
Recurso	Link
GitHub	https://github.com/feliperivas-ceo/sitm-mio-distributed-system.git

Cali, Colombia

Junio de 2026

Tabla de contenido

1. Resumen ejecutivo
 2. Contexto, alcance y objetivos
 3. Requerimientos y división inicial del trabajo
 4. Arquitectura general del sistema
 5. Patrones de arquitectura, diseño y distribución
 6. Estructura consolidada del repositorio
 7. Flujo operativo en tiempo real
 8. Dashboard, MVC, Observer y WebSocket
 9. Procesamiento histórico de velocidades
 10. Versión monolítica
 11. Versión concurrente
 12. Punto de distribución
 13. Versión distribuida Master-Worker
 14. Despliegue en laboratorio
 15. Resultados experimentales
 16. Problemas encontrados y refinamiento
 17. Discusión crítica
 18. Conclusiones y trabajo futuro
 19. Referencias
- Anexos

1. Resumen ejecutivo

Este informe documenta el proyecto SITM-MIO desarrollado en el curso de Arquitectura de Software. El trabajo no se limitó al cálculo histórico de velocidades: partió del diseño e implementación de un sistema distribuido de monitoreo para buses, sensores, datagramas, eventos y controladores de operación. Sobre esa base se incorporó posteriormente un experimento de analítica histórica con tres versiones: monolítica, concurrente y distribuida.

La arquitectura general integró comunicación remota con ICE, patrones Proxy y Broker, buses productores de datagramas, un servidor central consumidor, colas asíncronas, procesamiento mediante Thread Pool y Master-Worker, mensajería confiable para eventos críticos, persistencia, repositorios, dashboard bajo MVC, actualización de mapa mediante Observer y comunicación WebSocket. La estructura consolidada del repositorio suministrada por el equipo evidencia clases y paquetes asociados con estas responsabilidades.

Para el componente analítico se procesaron rutas activas y datagramas reales. La versión monolítica tardó 13.342 ms sobre datagrams-MiniPilot.csv; la versión concurrente, usando 12 hilos y chunks de 100.000 registros, tardó 8.068 ms y alcanzó un speedup de 1,65x. Finalmente, datagrams4Pilot.zip se descomprimió hasta alcanzar aproximadamente 67 GB y se procesó en laboratorio mediante un Master y dos Workers en modo streaming.

Alcance del informe: El documento diferencia entre patrones efectivamente representados en el código consolidado y patrones inicialmente contemplados como alternativa de diseño. Esta distinción evita atribuir como implementado aquello que no está respaldado por la estructura del proyecto.

2. Contexto, alcance y objetivos

2.1 Contexto del sistema

El SITM-MIO genera datagramas asociados con la operación de buses y rutas. Un bus puede reportar ubicación GPS, estado de puertas, estado del motor y eventos de prioridad alta o crítica. Para que el sistema sea útil, estos mensajes deben viajar desde los productores hasta un servidor central, procesarse sin bloquear la recepción, persistirse y reflejarse en una interfaz de monitoreo.

El proyecto se planteó como un ejercicio de refinamiento arquitectónico. Por esta razón, la solución combina patrones de diseño, tácticas de distribución y procesamiento de eventos. La fase final añadió una carga histórica masiva para evaluar el comportamiento del sistema ante volúmenes que superan el escenario de demostración en tiempo real.

2.2 Objetivo general

Diseñar, implementar y evaluar una arquitectura distribuida para el monitoreo y análisis del SITM-MIO, integrando patrones de comunicación remota, procesamiento asíncrono, visualización y distribución de cargas históricas.

2.3 Objetivos específicos

- Implementar buses simulados capaces de generar datagramas y eventos a partir de sensores.
- Utilizar ICE como middleware de comunicación remota entre buses y servidor central.
- Desacoplar recepción y procesamiento mediante Producer-Consumer y colas asíncronas.
- Procesar múltiples eventos utilizando Thread Pool y una coordinación Master-Worker.
- Aplicar mensajería confiable para eventos críticos mediante confirmaciones de recepción.
- Organizar la interfaz de monitoreo con MVC y actualizar el mapa mediante Observer y WebSocket.
- Calcular velocidades promedio por ruta y mes con versiones monolítica, concurrente y distribuida.
- Analizar resultados y documentar problemas reales de despliegue y refinamiento.

3. Requerimientos y división inicial del trabajo

Durante la planeación inicial, el equipo dividió la implementación en tres frentes. Esta distribución permitió avanzar en paralelo y asociar cada bloque funcional con los patrones más adecuados.

Persona	Responsabilidades funcionales	Patrones asignados
Felipe	Buses, sensores, datagramas, eventos, conductor e integración ICE.	Proxy, Broker y Producer.
Samuel	Servidor central, consumo de eventos, cola, procesamiento paralelo, mensajería confiable y coordinación de workers.	Consumer, Async Queue, Thread Pool, Master-Worker y Reliable Messaging.
Valentina Arana	Dashboard, controladores, mapa, analítica y actualización visual.	MVC, Observer y Fork-Join como alternativa para cálculos históricos.

La siguiente justificación resume la intención arquitectónica que guió el desarrollo:

Justificación de diseño: Para el desarrollo del sistema SITM-MIO se implementó una arquitectura distribuida utilizando Broker y Proxy mediante ICE, permitiendo que buses y estaciones enviaran datagramas y eventos al servidor central de manera desacoplada y remota. Los buses actuaron como productores y el servidor central como consumidor. Producer-Consumer y Async Queue permitieron manejar el volumen continuo de eventos. Thread Pool habilitó procesamiento paralelo. MVC organizó la interfaz de monitoreo y Observer permitió actualizar el mapa cuando llegaron nuevas posiciones GPS.

Mapa de patrones y responsabilidades

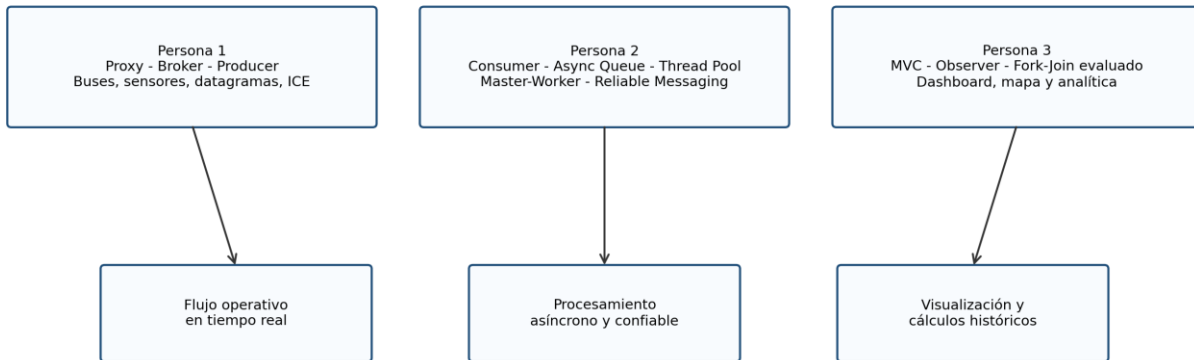


Figura 1. Distribución inicial de responsabilidades y patrones.

4. Arquitectura general del sistema

La solución consolidada combina componentes de simulación, dominio, comunicación, procesamiento, persistencia, repositorios, interfaz y analítica. El sistema se diseñó para desacoplar el envío de datagramas de su tratamiento posterior.

Arquitectura general del SITM-MIO implementado

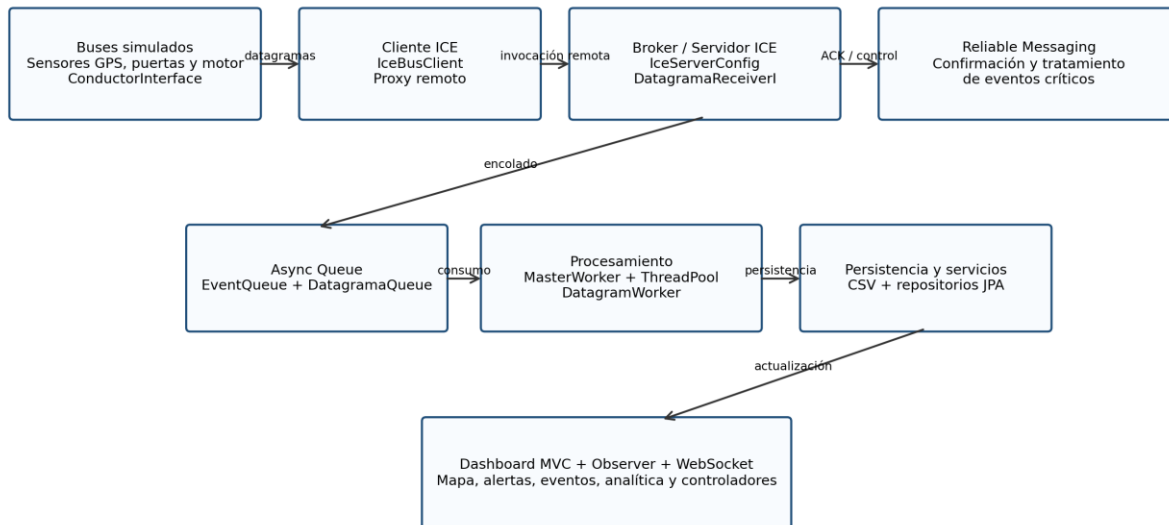


Figura 2. Arquitectura general implementada para el SITM-MIO.

4.1 Componentes principales

Componente	Responsabilidad
------------	-----------------

Componente	Responsabilidad
Buses simulados y sensores	Generar datagramas GPS, puertas, motor y eventos operativos.
IceBusClient	Actuar como Proxy del servicio remoto y enviar mensajes al servidor central.
IceServerConfig y DatagramaReceiverI	Exponer el receptor remoto y recibir invocaciones ICE.
EventQueue y DatagramaQueue	Desacoplar recepción y procesamiento mediante cola asíncrona.
MasterWorker, ThreadPoolProcessor y DatagramWorker	Consumir datagramas y procesarlos concurrentemente.
ReliableMessaging	Tratar confirmaciones y confiabilidad para eventos críticos.
Repositorios y CsvPersistence	Persistir información operacional e histórica.
Dashboard, servicios y WebSocket	Mostrar información de operación, mapa, eventos y analítica.
analytics.speed	Calcular velocidades promedio por ruta y mes en varias estrategias.

5. Patrones de arquitectura, diseño y distribución

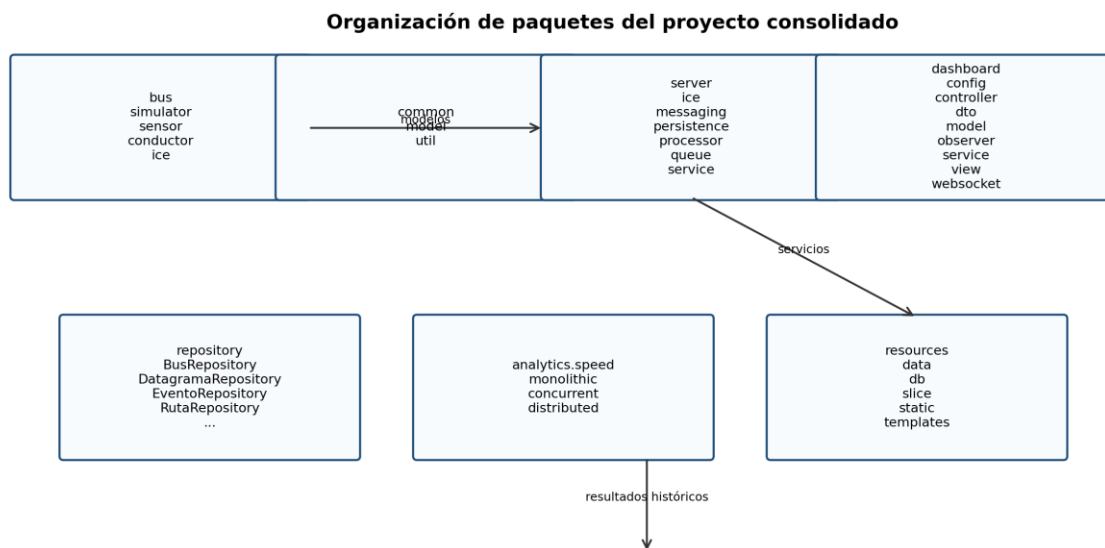
Los patrones no se utilizaron de forma aislada. Cada uno responde a un problema concreto del sistema: comunicación remota, desacoplamiento, concurrencia, confiabilidad, visualización o escalabilidad histórica.

Patrón	Problema que resuelve	Evidencia en la estructura
Proxy	Oculta los detalles de la comunicación remota. IceBusClient representa el acceso del bus al receptor remoto.	bus/ice/IceBusClient.java
Broker	ICE intermedia entre clientes y servidor, evitando acoplamiento directo.	server/ice/IceServerConfig.java, DatagramaReceiverI.java, slice/SitmMio.ice
Producer-Consumer	Los buses producen datagramas y el servidor los consume de forma independiente.	bus/simulator, server/queue, server/processor
Async Queue	Evita bloquear la recepción mientras se procesa cada datagrama.	EventQueue.java y DatagramaQueue.java
Thread Pool	Atiende múltiples trabajos mediante un conjunto reutilizable de hilos.	ThreadPoolProcessor.java
Master-Worker	Un coordinador delega procesamiento a workers; también se aplicó en el experimento distribuido histórico.	MasterWorker.java y analytics/speed/distributed
Reliable Messaging	Permite manejar confirmaciones o tratamiento especial para eventos críticos.	server/messaging/ReliableMessaging.java
MVC	Separa controladores, servicios, vistas y modelos del dashboard.	dashboard/controller, dashboard/service, dashboard/view
Observer	Propaga actualizaciones de posición hacia el mapa o componentes interesados.	dashboard/observer

Patrón	Problema que resuelve	Evidencia en la estructura
WebSocket	Soporta actualización en tiempo real del monitoreo.	dashboard/config/WebSocketConfig.java y dashboard/websocket
Fork-Join	Se consideró para dividir cálculos históricos; el benchmark final local quedó implementado con Thread Pool y chunks.	Alternativa evaluada, no evidenciada como clase final específica.

6. Estructura consolidada del repositorio

La estructura final del repositorio refleja la separación de responsabilidades descrita. La siguiente figura sintetiza los paquetes más relevantes observados en el árbol entregado por el equipo.



La estructura refleja separación de responsabilidades entre dominio, infraestructura, visualización y analítica.

Figura 3. Organización de paquetes del proyecto consolidado.

6.1 Paquete bus

Incluye simuladores, sensores, interfaz de conductor y cliente ICE. Entre las clases presentes se encuentran BusSimulatorRunner, BusSimulator, MultiBusSimulator, EventGenerator, SensorGPS, SensorPuertas, SensorMotor, ConductorInterface e IceBusClient.

6.2 Paquete common

Centraliza modelos compartidos como Bus, Ruta, Estacion, Datagrama, Evento, TipoEvento, Prioridad, Usuario, Rol y Zona. Esta decisión evita duplicar representaciones entre productores, servidor y dashboard.

6.3 Paquete server

Reúne el servidor central, recepción ICE, mensajería confiable, persistencia CSV, procesadores, colas y servicios. Las clases MasterWorker, ThreadPoolProcessor, DatagramWorker, GPSProcessor, EventQueue y DatagramQueue materializan el flujo asíncrono.

6.4 Paquete dashboard

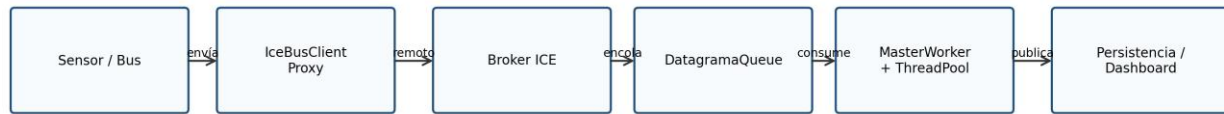
Contiene configuración, controladores, DTO, modelos, observadores, servicios, vistas y componentes WebSocket. Esta organización respalda el patrón MVC y la actualización de información en tiempo real.

6.5 Paquete analytics.speed

Se añadió para responder al requerimiento histórico de velocidades promedio. Incluye clases monolíticas, concurrentes y distribuidas, sin mezclar esta responsabilidad con el procesamiento operacional en tiempo real.

7. Flujo operativo en tiempo real

Flujo operativo de un datagrama



Eventos GPS, puertas, motor y emergencias se desacoplan de su procesamiento mediante cola asíncrona.

Figura 4. Recorrido simplificado de un datagrama desde el bus hasta su persistencia o visualización.

El bus o sensor genera un datagrama. IceBusClient realiza la invocación remota hacia el receptor expuesto por ICE. DatagramaReceiverI recibe el mensaje y lo encola. Posteriormente, MasterWorker consume la cola y delega el procesamiento a ThreadPoolProcessor. Finalmente, los resultados pueden persistirse y reflejarse en el dashboard.

7.1 Ventaja del desacoplamiento

La cola evita que un aumento temporal en el número de datagramas bloquee inmediatamente la recepción. El sistema puede absorber ráfagas y procesarlas conforme los workers estén disponibles. Esta separación mejora disponibilidad y escalabilidad.

7.2 Eventos críticos y confiabilidad

Los eventos críticos requieren tratamiento especial. ReliableMessaging representa la preocupación por confirmar la recepción y evitar que un evento importante pase inadvertido. El objetivo no es procesar todos los mensajes con el mismo costo, sino aplicar confiabilidad donde el riesgo operacional lo justifica.

8. Dashboard, MVC, Observer y WebSocket

El sistema incluye una interfaz de monitoreo para visualizar mapa, buses, alertas y eventos. La estructura dashboard separa controladores, servicios, modelos, vistas y observadores. Esto reduce el acoplamiento entre lógica de presentación y lógica operacional.

Capa	Elementos observados	Función
Controller	LoginController, MapController, AnalyticsController, DashboardController, EventosController, ZonaController, entre otros.	Recibir solicitudes y coordinar respuestas.

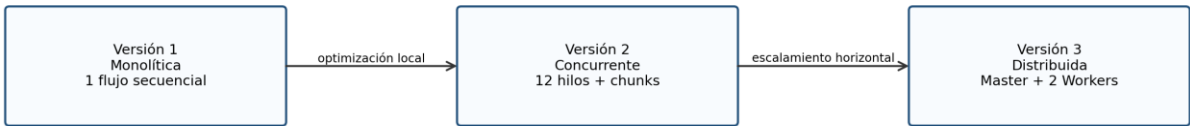
Capa	Elementos observados	Función
Service	AuthService, AnalyticsService, BusTrackingService, ControladorOperacionService, PublicService.	Centralizar lógica de aplicación.
View	DashboardView, LoginView, MapView y recursos HTML/JS/CSS.	Presentar información al usuario.
Observer	BusObserver, BusSubject, BusPositionObserver, MapUpdater.	Actualizar componentes cuando cambia una posición.
WebSocket	WebSocketConfig y BusWebSocketHandler.	Propagar actualizaciones en tiempo real.

La actualización del mapa no debería depender de consultas manuales constantes. Observer permite que un cambio de posición sea comunicado a los interesados. WebSocket complementa este mecanismo al mantener un canal adecuado para actualizaciones desde el backend hacia el navegador.

9. Procesamiento histórico de velocidades

Después de consolidar la arquitectura operacional, el profesor solicitó un experimento específico: calcular velocidades promedio por ruta y por mes para todas las rutas activas del piloto. El objetivo fue implementar una versión monolítica, una concurrente, determinar el punto donde vale la pena distribuir y construir una versión distribuida.

Evolución del cálculo histórico de velocidades



La distribución se adopta después de identificar límites prácticos de memoria y volumen de datos.

Figura 5. Evolución del componente analítico histórico.

9.1 Archivos suministrados

Archivo	Propósito
lines-241-ActiveGT.csv	Listado de rutas activas dentro del subconjunto piloto.
datagrams-MiniPilot.csv	Dataset reducido para validar solución completa y correcta.
datagrams4Pilot.zip / datagrams4Pilot.csv	Dataset masivo para evaluar necesidad de distribución.
Diccionario_De_Datos-OkGTM.pdf	Descripción de columnas y significado de variables.

9.2 Variables utilizadas

Variable	Uso
busId	Relacionar datagramas consecutivos del mismo bus.
lineId	Identificar la ruta para agrupar resultados.
odometer	Calcular distancia recorrida entre datagramas.
datagramDate	Calcular diferencia temporal y extraer el mes.

9.3 Fórmula

$$\text{velocidad (km/h)} = (\text{diferencia de odómetro} / \text{diferencia de tiempo en segundos}) \times 3,6$$

Se descartaron diferencias temporales no positivas, diferencias de odómetro no positivas y velocidades fuera del rango válido definido por la implementación. Posteriormente, los resultados se agruparon por ruta y mes.

10. Versión monolítica

La primera versión se implementó con procesamiento secuencial. Su función principal fue validar la lectura de archivos, el filtrado de rutas activas, la interpretación de columnas y el cálculo de promedios.

Clase	Responsabilidad
ActiveRoutesLoader	Carga las rutas activas desde lines-241-ActiveGT.csv.
SpeedCsvParser	Interpreta cada línea de datagrams-MiniPilot.csv.
SpeedRecord	Representa bus, ruta, odómetro y timestamp.
SpeedResult	Acumula suma, conteo y promedio por ruta y mes.
SpeedMonolithicCalculator	Ejecuta la lógica secuencial y escribe el CSV final.

Métrica	Resultado
Rutas activas cargadas	111
Resultados generados	97
Tiempo	13.342 ms
Archivo	data/output/velocidad_promedio_monolitico.csv

```
routeId,month,averageSpeedKmh,count
131,2019-05,35.18,18132
140,2019-05,42.42,16095
2102,2019-05,32.79,8097
```

11. Versión concurrente

La segunda versión reutilizó la lógica del cálculo, pero distribuyó bloques de líneas entre múltiples hilos. Esta estrategia permitió aprovechar mejor los núcleos disponibles en el equipo local.

Métrica	Resultado
Resultados generados	97
Hilos usados	12
Chunk size	100.000
Tiempo	8.068 ms
Speedup	1,65x
Archivo	data/output/velocidad_promedio_concurrente.csv

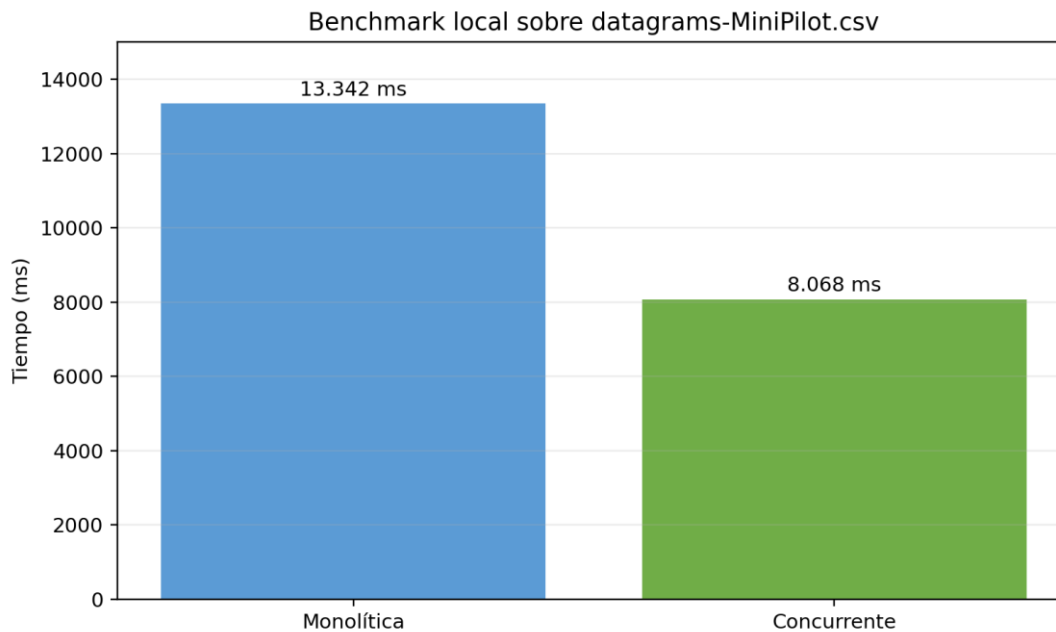


Figura 6. Comparación local entre versión monolítica y concurrente.

11.1 Interpretación

La mejora fue real, pero no lineal con respecto al número de hilos. Esto es esperable porque la solución incluye lectura de disco, parsing de texto, creación de objetos y consolidación de resultados. No todo el trabajo puede paralelizarse y parte del tiempo está condicionado por entrada y salida.

11.2 Validación funcional

La versión concurrente produjo los mismos 97 resultados que la versión monolítica y coincidió en los valores revisados. Esta equivalencia permitió validar que la optimización no alteró la lógica de negocio.

12. Punto a partir del cual vale la pena distribuir

El archivo datagrams4Pilot.zip se encontraba en el servidor 192.168.131.103. Al descomprimirlo en una carpeta del usuario se obtuvo un CSV de aproximadamente 67 GB. Este tamaño cambió las condiciones del problema: ya no era razonable depender únicamente de memoria y CPU de una sola máquina.

Aspecto	MiniPilot	datagrams4Pilot
Escala	Reducida	Aproximadamente 67 GB descomprimido
Estrategia suficiente	Monolítica o concurrente	Distribuida
Riesgo de memoria	Controlado	Alto sin streaming
Necesidad de particionar	No	Sí
Objetivo principal	Validar exactitud	Evaluar escalabilidad

El experimento no definió un umbral exacto en número de líneas porque no se ejecutó una batería completa de tamaños intermedios. Sin embargo, sí permitió establecer un punto práctico: MiniPilot puede resolverse localmente; un archivo de 67 GB justifica distribución por tiempo, memoria y operabilidad.

13. Versión distribuida con Master-Worker

Para la versión distribuida se mantuvo el patrón Master-Worker. El servidor de datos alojó el archivo original. El archivo se dividió en dos particiones aproximadas de 34 GB. Cada Worker procesó su fragmento en modo streaming y generó un CSV parcial. El Master consolidó ambos archivos.

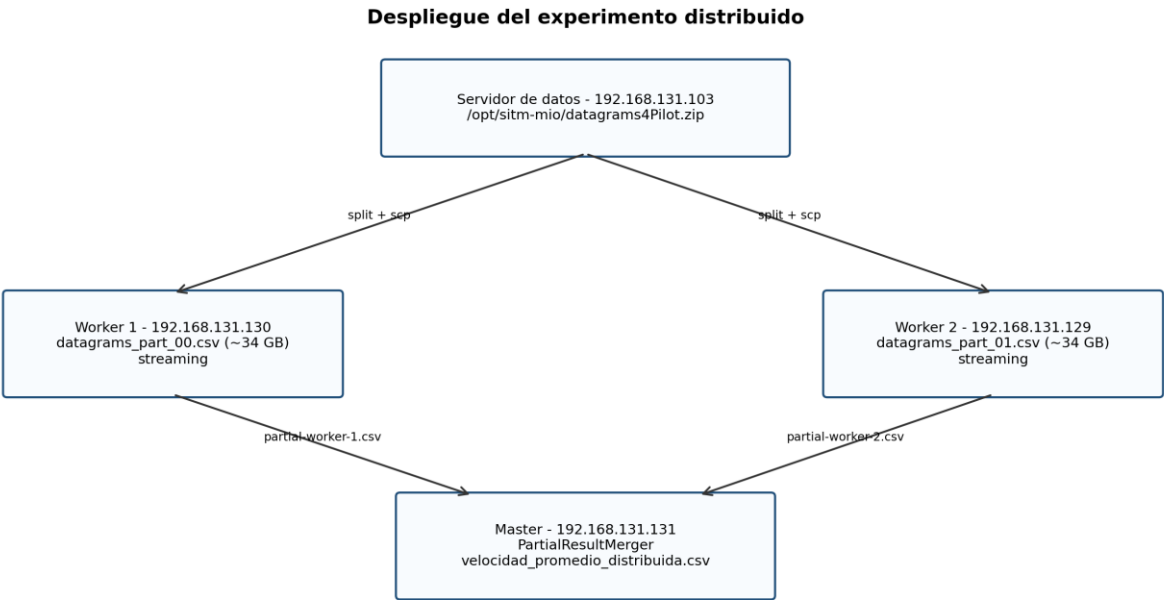


Figura 7. Despliegue real utilizado en laboratorio.

Nodo	IP	Rol
Servidor de datos	192.168.131.103	Almacenar y descomprimir datagrams4Pilot.zip.

Nodo	IP	Rol
Master	192.168.131.131	Consolidar archivos parciales.
Worker 1	192.168.131.130	Procesar datagrams_part_00.csv.
Worker 2	192.168.131.129	Procesar datagrams_part_01.csv.

13.1 Módulo distribuido

Clase	Responsabilidad
DistributedSpeedMaster	Generar la división lógica del trabajo y preparar comandos de workers.
DistributedSpeedWorker	Procesar una partición en streaming y generar acumulados parciales.
PartialSpeedResult	Representar suma de velocidades y cantidad por ruta y mes.
PartialResultMerger	Unir parciales y calcular el promedio consolidado.

13.2 Refinamiento hacia streaming

La primera versión del worker acumulaba listas de SpeedRecord y produjo OutOfMemoryError. La implementación se corrigió para conservar únicamente el último registro por combinación bus-ruta. De esta forma, cada nueva línea puede actualizar el acumulado sin cargar todo el archivo en memoria.

14. Despliegue y ejecución paso a paso en laboratorio

14.1 Preparación local

```
.\gradlew clean build
```

14.2 Transferencia de JAR plain

```
scp build\libs\sitm-mio-system-0.0.1-SNAPSHOT-plain.jar swarch@192.168.131.130:~/Desktop/Mio/data/output/
scp build\libs\sitm-mio-system-0.0.1-SNAPSHOT-plain.jar swarch@192.168.131.129:~/Desktop/Mio/data/output/
scp build\libs\sitm-mio-system-0.0.1-SNAPSHOT-plain.jar swarch@192.168.131.131:~/Desktop/MioMaster/data/output/
```

14.3 Preparación del dataset

```
ssh swarch@192.168.131.103
mkdir -p ~/Desktop/Mio/data/input
cd ~/Desktop/Mio/data/input
unzip /opt/sitm-mio/datagrams4Pilot.zip
mkdir -p parts
split -n 1/2 -d --additional-suffix=.csv datagrams4Pilot.csv parts/datagrams_part_
```

14.4 Copia de particiones

```
scp parts/datagrams_part_00.csv swarch@192.168.131.130:~/Desktop/Mio/data/input/
scp parts/datagrams_part_01.csv swarch@192.168.131.129:~/Desktop/Mio/data/input/
```

14.5 Ejecución de workers

```
/usr/lib/jvm/java-17-openjdk-amd64/bin/java \
-cp sitm-mio-system-0.0.1-SNAPSHOT-plain.jar \
com.sitmmio.analytics.speed.distributed.DistributedSpeedWorker \
/home/swarch/Desktop/Mio/data/input/datagrams_part_00.csv \
<RUTAS_ASIGNADAS> partial-worker-1.csv
```

14.6 Merge

```
/usr/lib/jvm/java-17-openjdk-amd64/bin/java \
-cp sitm-mio-system-0.0.1-SNAPSHOT-plain.jar \
com.sitmmio.analytics.speed.distributed.PartialResultMerger \
final/velocidad_promedio_distribuida.csv \
partial-worker-1.csv partial-worker-2.csv
```

15. Resultados experimentales

15.1 Benchmark local

Versión	Dataset	Tiempo	Resultados
Monolítica	datagrams-MiniPilot.csv	13.342 ms	97
Concurrente	datagrams-MiniPilot.csv	8.068 ms	97

15.2 Ejecución distribuida

Métrica	Worker 1	Worker 2
Líneas leídas	402.913.998	403.486.775
Registros válidos	194.545.095	195.032.932
Velocidades calculadas	18.203.488	23.208.751
Resultados parciales	369	339
Tiempo	212.652 ms	253.800 ms

Métrica de consolidación	Resultado
Archivos parciales unidos	2
Resultados finales	708

Métrica de consolidación	Resultado
Tiempo de merge	95 ms
Archivo final	final/velocidad_promedio_distribuida.csv

```
routeId,month,averageSpeedKmh,count
-1,2018-11,12.43,141566
-1,2018-12,12.52,1319314
131,2018-11,35.28,19292
131,2018-12,34.87,159625
```

16. Problemas encontrados y refinamiento arquitectónico

Problema	Evidencia	Solución aplicada
JAR no encontraba clase auxiliar	ClassNotFoundException al ejecutar DistributedSpeedMaster.	Usar el JAR plain para ejecutar clases con java -cp.
Java incorrecto	UnsupportedClassVersionError: versión 61 frente a 55.	Invocar explícitamente OpenJDK 17.
Disco lleno	No space left on device en un nodo.	Reducir despliegue y seleccionar workers con espacio suficiente.
Dataset no visible en workers	No such file or directory para /opt/sitm-mio/datagrams4Pilot.csv.	Descomprimir en servidor 103, dividir y transferir particiones.
Memoria insuficiente	OutOfMemoryError: Java heap space.	Reescribir DistributedSpeedWorker en modo streaming.
Caída nativa JVM	SIGSEGV durante lectura.	Reiniciar con implementación streaming y configuración estable.
Salida final sin carpeta	NullPointerException al crear directorios.	Escribir dentro de final/.

16.1 Evidencia textual seleccionada

```
java.lang.UnsupportedClassVersionError: class file version 61.0, this version of the Java Runtime only recognizes class file versions up to 55.0
```

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
```

```
mkdir: cannot create directory: No space left on device
```

Estos errores no fueron incidentes aislados: permitieron refinar la arquitectura. El paso hacia streaming y la selección cuidadosa de nodos surgieron directamente de la experimentación.

17. Discusión crítica

17.1 Aportes de la arquitectura operacional

El uso combinado de ICE, Proxy y Broker permite desacoplar buses y servidor. Producer-Consumer y Async Queue evitan que la recepción dependa del tiempo de procesamiento. Thread Pool mejora capacidad de atención. MVC, Observer y WebSocket organizan la presentación y actualización del dashboard.

17.2 Aportes del experimento histórico

El experimento mostró que no toda mejora requiere distribución inmediata. Para MiniPilot, la concurrencia local fue suficiente y redujo el tiempo en 1,65x. La distribución se justificó al aumentar la carga hasta 67 GB.

17.3 Control de exactitud

Las versiones monolítica y concurrente coincidieron en resultados. Para la versión distribuida debe realizarse una validación adicional antes de afirmar equivalencia completa. En la prueba ejecutada se dividió el archivo y también se asignaron subconjuntos de rutas por worker. Si una ruta aparece en ambas particiones, pueden omitirse registros. La siguiente iteración debería procesar todas las rutas por partición o particionar previamente por busld o lineld.

Observación académica: La ejecución distribuida demuestra escalabilidad y funcionamiento del patrón Master-Worker. La validación adicional propuesta fortalece la exactitud del resultado y evita presentar como definitivo un cálculo que todavía debe compararse contra una línea base completa.

18. Conclusiones y trabajo futuro

18.1 Conclusiones

1. El sistema consolidado integra patrones complementarios que responden a problemas reales de comunicación, procesamiento, confiabilidad y visualización.
2. ICE materializa la comunicación remota y permite representar Proxy y Broker sin acoplar directamente productores y servidor.
3. Producer-Consumer, Async Queue y Thread Pool permiten absorber datagramas y procesarlos sin bloquear la recepción.
4. MVC, Observer y WebSocket organizan el dashboard y facilitan actualización del mapa en tiempo real.
5. La versión concurrente del cálculo histórico mantuvo los resultados y redujo el tiempo local de 13.342 ms a 8.068 ms.
6. El dataset de 67 GB justificó el escalamiento horizontal y el uso de Master-Worker distribuido.
7. El procesamiento streaming fue decisivo para resolver límites de memoria.
8. La experimentación reveló que una arquitectura debe refinarse con evidencia, no solo con decisiones teóricas.
9. La salida distribuida requiere una validación adicional de particionado para asegurar exactitud completa.

18.2 Trabajo futuro

- Automatizar la asignación de tareas y la recolección de parciales por medio de ICE.
- Particionar por busld o lineld para preservar continuidad semántica y exactitud.
- Agregar balanceo dinámico de carga y tolerancia a fallos.
- Registrar métricas de CPU, RAM, I/O y red por nodo.
- Ejecutar tamaños intermedios para estimar un umbral cuantitativo de distribución.

- Integrar resultados históricos con el dashboard.
- Agregar pruebas automáticas de equivalencia entre versiones.

19. Referencias

- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P. y Stal, M. (1996). Pattern-Oriented Software Architecture: A System of Patterns. Wiley.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. (2011). Distributed Systems: Concepts and Design. Addison-Wesley.
- Oracle. (s. f.). Java Platform, Standard Edition 17 API Specification: ExecutorService y ThreadPoolExecutor.
- ZeroC. (s. f.). Ice documentation: proxies, object adapters y Slice.
- Spring. (s. f.). Spring Framework documentation: WebSocket y STOMP messaging.

Anexo A. Resumen de clases por módulo

Módulo	Clases o submódulos principales
bus/simulator	BusSimulatorRunner, BusSimulator, MultiBusSimulator, EventGenerator
bus/sensor	Sensor, SensorGPS, SensorMotor, SensorPuertas
bus/conductor	ConductorInterface
bus/ice	IceBusClient
common/model	Bus, Ruta, Estacion, Datagrama, Evento, TipoEvento, Prioridad, Usuario, Rol, Zona
server/ice	DatagramaReceiverI, IceServerConfig
server/messaging	ReliableMessaging
server/queue	EventQueue, DatagramaQueue
server/processor	DatagramWorker, EventProcessor, GPSProcessor, MasterWorker, ProcessingManager, ThreadPoolProcessor
dashboard	config, controller, dto, model, observer, service, view, websocket
analytics/speed	ActiveRoutesLoader, SpeedCsvParser, SpeedRecord, SpeedResult, SpeedMonolithicCalculator, SpeedConcurrentCalculator, SpeedBenchmarkRunner
analytics/speed/distributed	DistributedSpeedMaster, DistributedSpeedWorker, PartialSpeedResult, PartialResultMerger

Anexo B. Evidencias recomendadas en el repositorio

```
evidence/distributed-speed-benchmark/
├── master/
│   └── velocidad_promedio_distribuida.csv
├── worker1/
│   └── partial-worker-1.csv
└── worker2/
```

```
| partial-worker-2.csv  
| README.md
```

Los datasets de 34 GB y 67 GB no deben versionarse en Git. El repositorio debe conservar código, resultados parciales livianos, resultado final y documentación de ejecución.

Anexo C. Comandos locales del benchmark

```
cd "C:\Users\muril\Documents\GitHub\sitm-mio-distributed-system\sitm-mio-system"  
& "C:\Program Files\Java\jdk-17\bin\java.exe" -cp "build\classes\java\main;build\resources\main"  
com.sitmio.analytics.speed.SpeedBenchmarkRunner
```

Anexo D. Nota sobre capturas

No se conservaron capturas de pantalla originales de todas las ejecuciones. Por esta razón, el informe incorpora transcripciones textuales de terminal y tablas con resultados registrados.

